

TorusNet 4.0 Implémentation

L'Architech (Gabriel Breton-Breault)

◆ Introduction — Langage principal du protocole : C

La programmation de TorusNet repose entièrement sur le **langage C**, qui constitue la base technique du protocole. Ce choix n'est pas anodin : il répond directement aux exigences structurelles, directionnelles et matérielles du projet. TorusNet manipule des volumes binaires massifs, traverse des structures géométriques internes, et exige un contrôle précis de la mémoire — des caractéristiques qui rendent le C particulièrement adapté.

Le protocole doit gérer des éléments tels que des seeds binaires amplifiés, des vecteurs directionnels de grande taille, des structures internes du tore privé, des portails directionnels et des buffers de cohérence. Le C offre un accès direct aux pointeurs, aux allocations, aux structures, aux blocs binaires et aux accès mémoire séquentiels, ce qui est indispensable pour un protocole fondé sur une architecture directionnelle.

Au-delà du contrôle mémoire, le C garantit une **performance brute** essentielle. TorusNet doit amplifier un seed de 30 bits en une séquence pouvant atteindre plusieurs centaines de mégaoctets, traverser un tore privé, valider la cohérence topologique et générer des clés directionnelles uniques à chaque requête. Le langage C permet une exécution rapide, une latence minimale et une gestion efficace des buffers, tout en restant compatible avec des implémentations matérielles futures.

Ce choix s'inscrit également dans une vision à long terme : TorusNet est conçu pour être **photon-ready**. Les architectures photoniques et les processeurs directionnels reposent généralement sur des langages bas niveau tels que C ou C++. Le C facilite donc la transition vers des modules avancés comme la Forteresse de Verre ou les systèmes SMD/VSMD.

Enfin, TorusNet doit rester indépendant de toute surcouche logicielle. Il ne doit pas dépendre d'un framework, d'une machine virtuelle, d'un runtime ou d'un garbage collector. Le C, minimaliste et autonome, est parfaitement adapté à un protocole expérimental dont la sécurité repose sur la structure et la géométrie plutôt que sur la cryptographie classique.

Chapitre 1 — Génération aléatoire du tore privé

Le tore privé est une structure directionnelle unique composée de **1 000 000 de cases** générées aléatoirement. Il est stocké sur le disque dur sous forme de plusieurs fichiers chiffrés et constitue la base de la sécurité de TorusNet : non transmissible, non dérivable et impossible à reconstruire sans accès direct à ses fichiers internes.

Le tore privé repose sur trois éléments principaux :

1. **le fichier des bits normaux,**
2. **le fichier des déplacements directionnels et des sauts exponentiels,**
3. **les cases inatteignables (10 % du tore),** qui peuvent également servir de cases de départ.

Ces fichiers sont chargés en mémoire RAM lors de l'utilisation du protocole et sont **lus en parallèle** pendant la traversée du tore.

1.1 — Fichier des bits normaux (binaire pur)

Le fichier des bits normaux contient une séquence binaire pure représentant les **1 000 000 de cases** du tore. Il est généré aléatoirement et stocké sous forme chiffrée sur le disque dur.

Caractéristiques :

- fichier binaire pur,
- chiffré sur le disque,
- chargé en RAM lors de l'utilisation,
- utilisé pour la lecture directionnelle,
- **10 % des cases sont marquées comme inatteignables par saut exponentiel.**

Ces cases inatteignables jouent un rôle particulier : elles peuvent servir de **cases de départ**, car elles ne peuvent jamais être atteintes par un saut exponentiel linéaire. Elles forment donc des points d'origine directionnels impossibles à rejoindre autrement.

1.2 — Fichier des déplacements directionnels

Le second fichier contient les **4 déplacements possibles par case**, ainsi que les **sauts exponentiels** associés. Chaque entrée suit une structure compacte :

Code

$101\ 2^4, 010\ 3^5, 110\ 2^4, 111\ 3^5, \dots$

Interprétation :

- Les **3 bits** indiquent la **direction** :
 - haut
 - bas
 - gauche
 - droite
 - diagonales
- L'exposant indique le **saut exponentiel** à effectuer dans le tore.

Ce fichier est également :

- chiffré sur le disque dur,
- chargé en RAM lors de l'utilisation,
- lu en parallèle avec le fichier des bits normaux.

1.3 — Comportement torique du canevas

Le tore privé se comporte comme une surface torique. Les déplacements directionnels respectent les règles suivantes :

Déplacements :

- dépasser en haut → réapparaître en bas
- dépasser en bas → réapparaître en haut
- dépasser à gauche → réapparaître à droite
- dépasser à droite → réapparaître à gauche

Sauts exponentiels :

Les sauts exponentiels se font **de manière linéaire** dans le fichier des bits normaux :

- on avance de N^E positions,
- si on atteint la fin → on revient au début,
- le saut est indépendant de la direction.

1.4 — ID de case (hexadécimal)

Certaines cases du tore — notamment les **cases de départ** — possèdent un **ID directionnel** codé en **hexadécimal**. Cet ID encode :

- la direction,
- la nature de la case,
- son rôle dans la cohérence directionnelle.

Important :

- **Toutes les cases n'ont pas un ID.**
- Seules les **cases de départ** (dont les 10 % inatteignables) possèdent un identifiant.
- Les autres cases sont des cases internes sans ID.

1.5 — Résultat : un tore privé unique, non dérivable et inviolable

Grâce à cette génération :

- le tore contient **1 000 000 de cases**,
- 90 % sont atteignables par saut exponentiel,
- 10 % sont inatteignables et servent de points d'origine,
- les ID hexadécimaux définissent les cases de départ,
- les déplacements sont toriques,
- les sauts sont linéaires,
- les deux fichiers sont lus en parallèle,
- le tore est impossible à reconstruire sans ses fichiers internes.

Le tore privé devient ainsi une structure directionnelle autonome, non transmissible, et essentielle à la sécurité de TorusNet.

Chapitre 2 — Génération du seed, reconstruction du fichier et disparition du tore public

Dans TorusNet, le seed de **30 bits** n'est pas simplement un point d'origine : il est généré **pendant** la traversée du tore privé, en même temps que le fichier à encoder est reconstruit. Ce fonctionnement dynamique rend le tore public totalement obsolète dans les versions modernes du protocole.

Ce chapitre décrit comment :

1. le programme génère le seed,
2. reconstruit le fichier pendant la traversée,
3. utilise les bits amplifiés pour se déplacer dans le tore privé,
4. remonte jusqu'au seed,
5. encode la case d'arrivée,
6. produit la clé directionnelle finale.

2.1 — Génération du seed pendant la traversée

Contrairement aux versions anciennes du protocole, le seed n'est pas généré avant la traversée : il est produit **pendant** que le programme parcourt le tore privé et reconstruit le fichier.

Le seed final est donc directement lié :

- au chemin parcouru,
- aux bits normaux lus,
- aux bits fantômes utilisés,
- aux directions choisies,
- à la case d'arrivée.

Cette approche rend chaque seed **unique, non reproductible, et impossible à dériver** sans le tore privé.

2.2 — Amplification exponentielle : $30 \text{ bits}^{2^{28}} \approx 1 \text{ Go}$

Une fois le seed généré, il est amplifié selon la formule :

$$30 \text{ bits}^{2^{28}}$$

Cette amplification produit **environ 1 Go de bits directionnels**.

Ces bits amplifiés servent à :

- guider les déplacements dans le tore privé,
- alimenter les sauts exponentiels,
- reconstruire le fichier,

- générer les vecteurs de mouvement,
- structurer la cohérence topologique.

Le seed devient ainsi une **source directionnelle massive**, suffisante pour remplacer entièrement le tore public.

2.3 — Reconstruction du fichier pendant la traversée

Lorsqu'un fichier doit être encodé, le programme :

1. choisit une **case de départ**,
2. commence à parcourir le tore privé,
3. lit à chaque étape :
 - soit un **bit normal**,
 - soit un **bit fantôme**,
4. encode la direction choisie en binaire (3 bits),
5. ajoute le bit lu à la reconstruction du fichier.

La reconstruction se fait **en temps réel**, pendant la traversée.

Si le fichier reconstruit est **plus petit** que la quantité de bits générée par le seed amplifié, le programme **remplit le reste de l'espace** avec des 0 et des 1 aléatoires.

Ainsi, le fichier reconstruit atteint toujours la taille maximale que le seed peut fournir.

2.4 — Déplacements dans le tore privé grâce aux bits amplifiés

Les bits amplifiés (≈ 1 Go) servent directement à se déplacer dans le tore privé :

- ils déterminent les directions,
- ils alimentent les sauts exponentiels,
- ils guident la lecture des bits normaux ou fantômes,
- ils définissent la trajectoire complète.

Le tore privé devient un espace directionnel entièrement piloté par les bits amplifiés du seed.

2.5 — Remontée vers le seed de 30 bits

Une fois la reconstruction terminée, le programme **remonte** la séquence générée :

- des bits amplifiés,
- vers les bits originaux,
- jusqu'au seed de 30 bits.

Cette remontée permet de retrouver le **point d'origine directionnel**, indispensable pour générer la clé finale.

2.6 — Encodage de la case d'arrivée

La case d'arrivée dans le tore privé est également encodée dans la clé :

- son ID hexadécimal,
- sa position,
- sa direction finale.

Cela permet au serveur de vérifier :

- la cohérence du parcours,
- la validité du seed,
- l'intégrité de la clé.

2.7 — Disparition du tore public

Avec ce fonctionnement :

- le seed génère son propre espace directionnel,
- les bits amplifiés remplacent totalement le tore public,
- le tore privé fournit toute la structure,
- les fichiers internes définissent les règles,
- les sauts exponentiels assurent la dynamique.

Le tore public devient **inutile, redondant, et supprimé** dans TorusNet 4.0+.

2.8 — Clé directionnelle finale

La clé finale contient :

- le seed de 30 bits,
- les directions parcourues,

- les bits normaux et fantômes utilisés,
- la case d'arrivée,
- la cohérence topologique.

Elle est **unique, non reproductible, et impossible à dériver** sans :

- le tore privé,
- les fichiers internes,
- le seed amplifié,
- la trajectoire exacte.

Chapitre 3 — Déplacements directionnels, sauts exponentiels et navigation dans le tore privé

Le tore privé est un espace directionnel fermé composé de **1 000 000 de cases**, chacune définie par des bits normaux, des bits fantômes, des directions possibles et des sauts exponentiels. Ce chapitre décrit précisément comment le programme se déplace dans ce tore, comment les règles directionnelles sont appliquées, et pourquoi cette structure rend TorusNet impossible à dériver ou cartographier.

3.1 — Les deux sources de lecture : bits normaux et bits fantômes

Pendant la traversée du tore privé, le programme lit deux types de données :

Bits normaux

- proviennent du fichier binaire pur,
- représentent la matière brute du tore,
- sont utilisés pour reconstruire le fichier à encoder,
- sont présents dans **toutes les cases**.

Bits fantômes

- proviennent du fichier des déplacements,
- représentent les directions et les sauts exponentiels,
- ne sont pas des données du fichier reconstruit,
- servent uniquement à guider la navigation.

À chaque étape, le programme décide :

- de lire un bit normal,
- ou de lire un bit fantôme,
- ou de combiner les deux.

Cette lecture parallèle crée une trajectoire directionnelle unique.

3.2 — Les directions : 3 bits par déplacement

Chaque case du tore contient **4 directions possibles**, codées sur **3 bits** :

Code

000 = haut

001 = bas

010 = gauche

011 = droite

100 = diagonale haut-gauche

101 = diagonale haut-droite

110 = diagonale bas-gauche

111 = diagonale bas-droite

Ces directions sont lues dans le fichier des déplacements.

Le programme encode chaque direction en binaire dans le fichier reconstruit, ce qui permet au serveur de vérifier la cohérence du parcours.

3.3 — Comportement torique du tore privé

Le tore privé fonctionne comme une surface torique :

- dépasser en haut → réapparaître en bas,
- dépasser en bas → réapparaître en haut,
- dépasser à gauche → réapparaître à droite,
- dépasser à droite → réapparaître à gauche.

Ce comportement garantit :

- l'absence de bordures,
- une continuité parfaite,
- une impossibilité de déterminer la taille réelle du tore depuis l'extérieur.

3.4 — Les sauts exponentiels

Chaque direction est accompagnée d'un **saut exponentiel**, par exemple :

Code

101 2³⁴

010 3⁵

110 2⁴

L'exposant indique combien de cases le programme doit sauter **linéairement** dans le fichier des bits normaux.

Règles des sauts exponentiels :

- le saut est indépendant de la direction,
- il avance dans le fichier binaire pur,
- s'il atteint la fin → il revient au début,
- il ne peut jamais atteindre les **10 % de cases inatteignables**.

Ces cases inatteignables servent de **points d'origine** et de **cases de départ**.

3.5 — Cases inatteignables et cases de départ

Le tore contient :

- **90 % de cases atteignables**,
- **10 % de cases inatteignables** par saut exponentiel.

Les cases inatteignables :

- ne peuvent être atteintes que par déplacement directionnel,
- ne peuvent jamais être atteintes par un saut exponentiel,
- servent de **cases de départ**,
- possèdent un **ID hexadécimal**,

- sont utilisées pour générer des clés directionnelles uniques.

Les autres cases n'ont pas d'ID.

3.6 — Navigation directionnelle et reconstruction du fichier

Pendant la traversée :

1. le programme lit un bit normal ou fantôme,
2. choisit une direction,
3. encode cette direction en binaire,
4. applique le saut exponentiel,
5. ajoute le bit lu au fichier reconstruit.

Si le fichier reconstruit est plus petit que la capacité du seed amplifié, le programme remplit le reste avec des 0 et des 1 aléatoires.

3.7 — Arrivée, remontée et cohérence

À la fin de la traversée :

- la **case d'arrivée** est encodée dans la clé,
- le programme **remonte** les bits amplifiés jusqu'au seed de 30 bits,
- le seed devient la base de la clé directionnelle,
- la cohérence du parcours est vérifiée par le serveur.

3.8 — Résultat : une navigation impossible à reproduire

Grâce à cette structure :

- chaque parcours est unique,
- chaque clé est non reproductible,
- le tore privé est impossible à cartographier,
- les sauts exponentiels empêchent toute analyse,
- les cases inatteignables garantissent des origines directionnelles inviolables.

Le tore privé devient un espace directionnel autonome, impossible à dériver sans accès aux fichiers internes.